

Serial UART Communication Experiment

Serial UART Communication Experiment

- 1. Serial UART Communication Overview
- 2. Core Concepts
 - 2.1 UART Frame Format
 - 2.2 ESP32 Hardware UART
 - 2.3 Key APIs
 - 2.4 RX Pin Pull-up Timing Trap
- 3. Hardware Dependencies
- 4. Project Architecture and Flow
- 5. Source Code
 - 5.1 Macro Definitions and Global Variables
 - 5.2 `setup()` — Initialize Dual Serial Ports
 - 5.3 `loop()` — Transmit and Receive Loopback
- 6. Operation Flow
 - 6.1 Compile and Flash
 - 6.2 Normal Phenomena
 - 6.3 Verification Methods
- 7. Common Issues

1. Serial UART Communication Overview

UART is the most commonly used inter-board communication interface, requiring only TX/RX two wires for full-duplex transmission and reception. This experiment uses ESP32's hardware UART2 (Serial2), with TX (GPIO14) and RX (GPIO5) connected with Dupont wires to form a loopback. A string is sent every 2 seconds and the data sent by itself is received, demonstrating the complete usage of `Serial2.begin`, `write`, and `readBytes`.

Typical Application Scenarios:

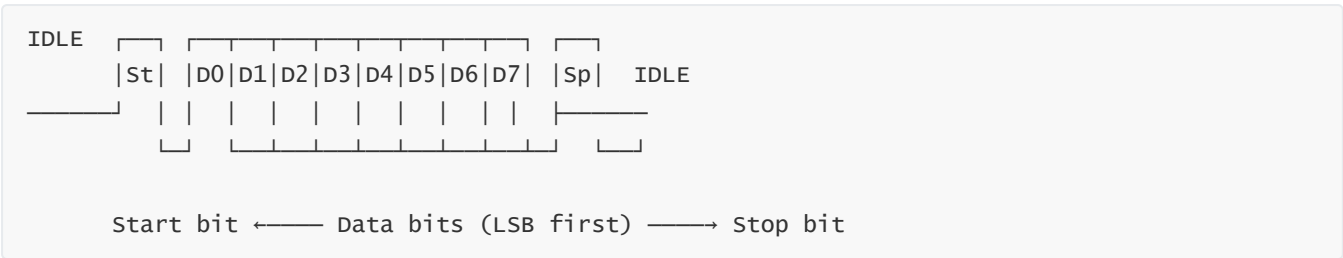
Application Type	Examples
Debug Output	Serial monitor, log printing
Module Communication	GPS module, 4G module, fingerprint module, Bluetooth AT commands
Industrial Bus	RS-232, RS-485 (with transceiver chip)
Inter-board Communication	Point-to-point data exchange between two MCUs

This project demonstrates UART2 loopback communication: the MCU sends a string via TX, RX receives the echoed data, achieving self-send and self-receive hardware verification.

2. Core Concepts

2.1 UART Frame Format

UART transmits data in frames (common 8N1 format = 8 data bits + no parity + 1 stop bit):



Parameter	Meaning	Common Values
Baud Rate	Symbols per second	9600, 115200, 921600
Data Bits	Data width	5 / 6 / 7 / 8
Parity	Parity check	None / Even / Odd
Stop Bits	Stop bit length	1 / 1.5 / 2

On the same bus, the sender and receiver must have **identical baud rate, data bits, parity, and stop bits**, otherwise garbled characters will appear.

2.2 ESP32 Hardware UART

The ESP32-S3 has 3 hardware UART controllers, supporting GPIO matrix remapping, with any pin configurable as UART function.

Arduino Object	ESP32 Hardware	Default TX/RX	Description
<code>Serial1</code>	UART0	GPIO43 / GPIO44	Connected to on-board USB-UART bridge chip
<code>Serial11</code>	UART1	Any GPIO	User-defined mapping
<code>Serial12</code>	UART2	Any GPIO	Used in this experiment, mapped to GPIO14/5

2.3 Key APIs

- `Serial12.begin(115200, SERIAL_8N1, RXD_PIN, TXD_PIN)` — Initialize UART2
- `Serial12.readBytes(buf, size)` — Read bytes from RX FIFO, more reliable than `readString()`
- `Serial12.flush()` — Wait for all data in TX FIFO to be sent

2.4 RX Pin Pull-up Timing Trap

`pinMode(RXD_PIN, INPUT_PULLUP)` must be set **before** `Serial2.begin()`. Once `begin()` is called, the RX pin is taken over by the hardware UART, and setting `INPUT_PULLUP` afterwards will be ineffective. When idle, a floating RX will cause noise data to be received.

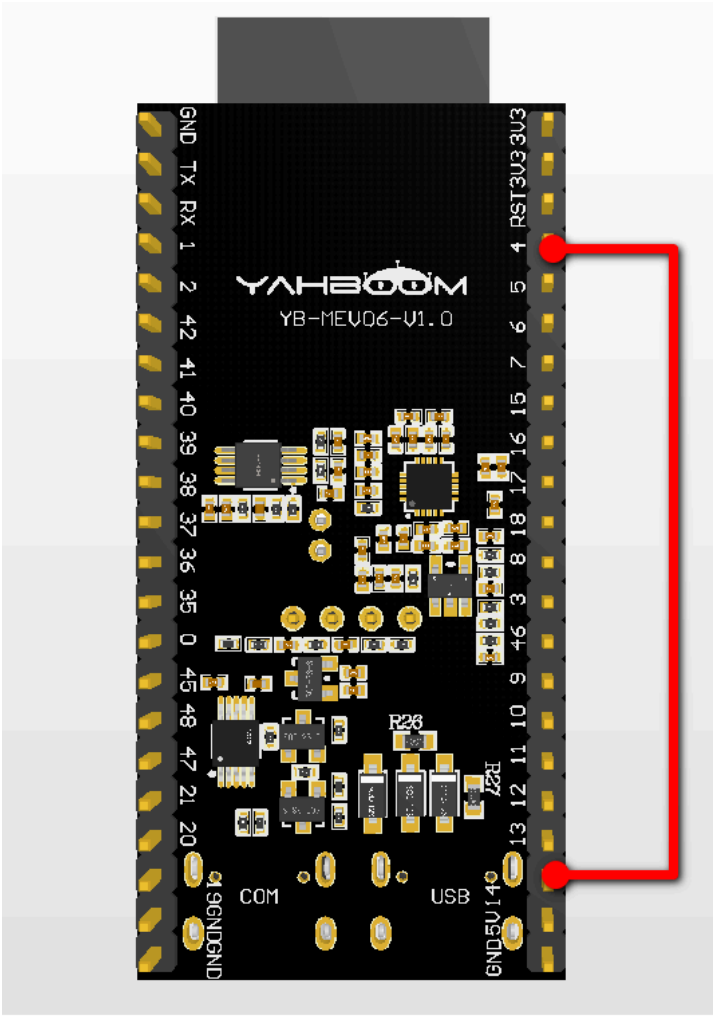
3. Hardware Dependencies

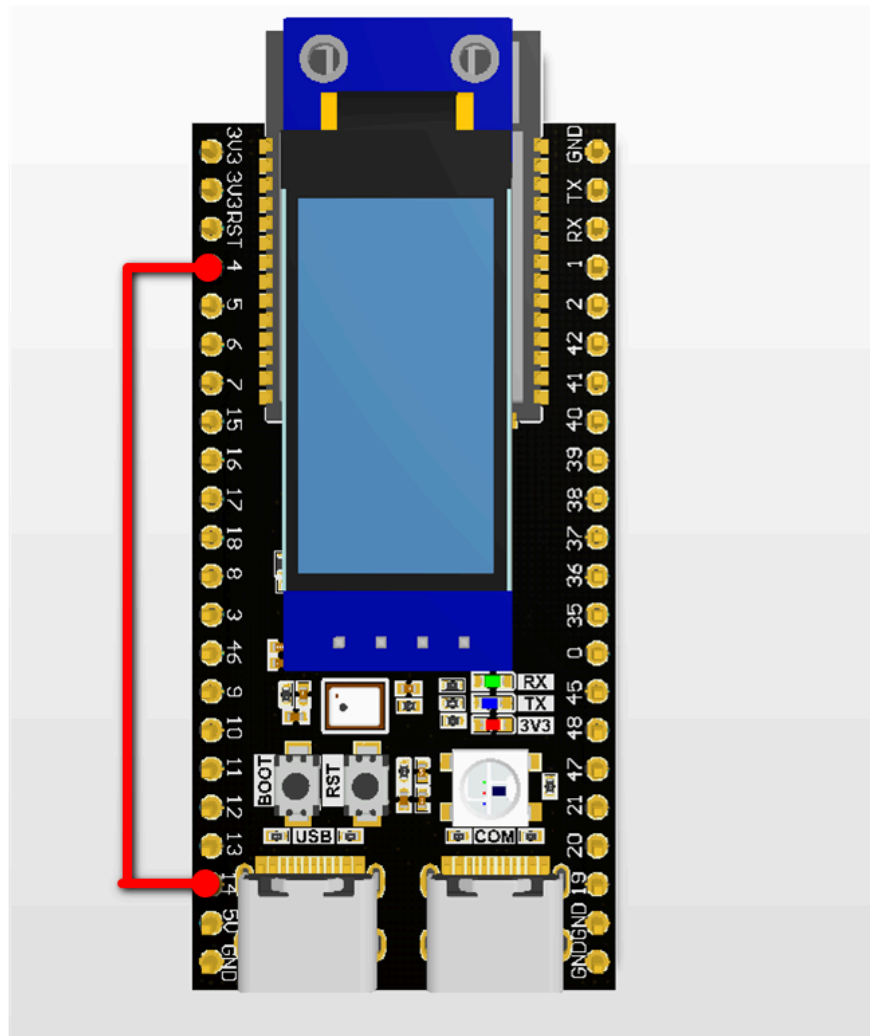
Pin	Peripheral	Configuration	Description
GPIO14	TXD	UART2 TX	Serial transmission
GPIO5	RXD	UART2 RX (<code>INPUT_PULLUP</code>)	Serial reception, must be pulled up before begin

ESP32-S3 UART Hardware Features:

Feature	Parameter
Number of UART controllers	3 (UART0 / UART1 / UART2)
Maximum baud rate	5 Mbps
FIFO depth	128 bytes (independent TX/RX)
Data width	5 / 6 / 7 / 8 bits
Parity	None / Even / Odd
Stop bits	1 / 1.5 / 2 bits
Hardware flow control	Supports RTS/CTS
GPIO mapping	Any GPIO can be configured as TX/RX/CTS/RTS via IO MUX

Connect GPIO14 and GPIO5 with a Dupont wire to form a "self-send and self-receive" loopback.





4. Project Architecture and Flow

```
setup()
├ Serial1.begin(115200)           // Monitor serial UART0
├ pinMode(RXD_PIN, INPUT_PULLUP) // Must be before Serial2.begin
├ Serial2.begin(115200, 8N1, RX, TX) // Initialize UART2
└ Print startup message

loop()
├ Serial2.write("GOOD GOOD STUDY...\r\n") // Send
├ Serial2.flush()                          // wait for FIFO to finish
├ Serial2.setTimeout(200)
├ len = Serial2.readBytes(rxBuf, 255)      // Receive loopback data
├ Filter non-printable characters → Print RX content via serial
└ delay(2000)                             // 2 second interval
```

5. Source Code

Source Code Location: `Source code → Arduino → 2.Serial communication → 1.UART → 1.UART.ino`

5.1 Macro Definitions and Global Variables

```
#define TXD_PIN 14
#define RXD_PIN 5
#define BUF_SIZE 256

uint8_t rxBuf[BUF_SIZE];
```

5.2 `setup()` — Initialize Dual Serial Ports

Configure the monitor serial port first, then initialize UART2 for loopback communication.

```
void setup() {
    Serial.begin(115200);

    // Important: Set RX pull-up before Serial2.begin()
    pinMode(RXD_PIN, INPUT_PULLUP);

    Serial2.begin(115200, SERIAL_8N1, RXD_PIN, TXD_PIN);
    Serial.printf("UART Started, TX=GPIO%d, RX=GPIO%d\n", TXD_PIN, RXD_PIN);
}
```

`pinMode(RXD_PIN, INPUT_PULLUP)` must be called before `Serial2.begin()`. When TX/RX are shorted with Dupont wires, pull-up can be omitted, but keeping it is good defensive programming practice.

5.3 `loop()` — Transmit and Receive Loopback

Send a string, then read it back from the same port. Filter non-printable characters to keep the monitor readable.

```
void loop() {
    Serial2.write("GOOD GOOD STUDY, DAY DAY UP!!!!!!\r\n");
    Serial2.flush(); // wait for FIFO to finish

    Serial2.setTimeout(200);
    int len = Serial2.readBytes(rxBuf, BUF_SIZE - 1);

    if (len > 4) {
        for (int i = 0; i < len; i++) {
            if (rxBuf[i] == '\r' || rxBuf[i] == '\n') continue;
            if (rxBuf[i] < 0x20 || rxBuf[i] > 0x7E) rxBuf[i] = '.';
        }
        rxBuf[len] = '\0';
        Serial.printf("[RX] %s\n", rxBuf);
    }

    delay(2000);
}
```

Filtering replaces non-printable characters with `.`. The `len > 4` threshold filters out noise.

6. Operation Flow

6.1 Compile and Flash

Arduino Flash Process: See 1. Before Development → 2. Build and Flash.

1. Short GPIO14 (TX) and GPIO5 (RX) with a Dupont wire
2. Open `1.UART.ino`

6.2 Normal Phenomena

Open the **Serial Monitor** (115200 bps), printing every 2 seconds:

```
[UART] UART Started, TX=GPI014, RX=GPI05
[UART] [TX] GOOD GOOD STUDY,DAY DAY UP!!!!!!
[UART] [RX] GOOD GOOD STUDY,DAY DAY UP!!!!!!

[UART] [TX] GOOD GOOD STUDY,DAY DAY UP!!!!!!
[UART] [RX] GOOD GOOD STUDY,DAY DAY UP!!!!!!

[UART] [TX] GOOD GOOD STUDY,DAY DAY UP!!!!!!
[UART] [RX] GOOD GOOD STUDY,DAY DAY UP!!!!!!

[UART] [TX] GOOD GOOD STUDY,DAY DAY UP!!!!!!
[UART] [RX] GOOD GOOD STUDY,DAY DAY UP!!!!!!
```

```
[UART] UART Started, TX=GPIO14, RX=GPIO5
[UART] [TX] GOOD GOOD STUDY, DAY DAY UP!!!!!!
[UART] [RX] GOOD GOOD STUDY, DAY DAY UP!!!!!!

[UART] [TX] GOOD GOOD STUDY, DAY DAY UP!!!!!!
[UART] [RX] GOOD GOOD STUDY, DAY DAY UP!!!!!!

[UART] [TX] GOOD GOOD STUDY, DAY DAY UP!!!!!!
[UART] [RX] GOOD GOOD STUDY, DAY DAY UP!!!!!!

[UART] [TX] GOOD GOOD STUDY, DAY DAY UP!!!!!!
[UART] [RX] GOOD GOOD STUDY, DAY DAY UP!!!!!!
```

After disconnecting or removing the Dupont wire, the print becomes:

[illegible]

6.3 Verification Methods

Operation	Expected Result
Power on after shorting TX/RX	TX and RX content are identical
Remove Dupont wire	RX has no output
Change <code>delay(2000)</code> to <code>delay(200)</code>	Transmit/receive frequency significantly increases

7. Common Issues

Phenomenon	Cause	Solution
RX has no output	TX/RX not shorted or poor contact	Check Dupont wire, confirm GPIO14→GPIO5
RX is all garbled <code>.</code>	Baud rate mismatch or floating noise	Confirm both are 115200, check <code>INPUT_PULLUP</code>
<code>INPUT_PULLUP</code> ineffective	Set after <code>begin()</code>	Move <code>pinMode</code> before <code>begin()</code>